# HW 3

Bryan Mui - UID 506021334

2025-02-12

## Section 1: Cook's relational data

library and read csv files:

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Rows: 5 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): name, Inventor
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 22 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): recipe, food_item
## dbl (1): weight (oz)
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 25 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): food_item, shop
## dbl (1): price (US dollars per lb)
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 11 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): item, type
```

```
## dbl (1): calories
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
## Rows: 5 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): name, Inventor
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
## Rows: 22 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): recipe, food_item
## dbl (1): weight (oz)
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
## Rows: 25 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): food_item, shop
## dbl (1): price (US dollars per lb)
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
## Rows: 11 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): item, type
## dbl (1): calories
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
```

**1a) Write R code to return the types of food and the total calories for each type to make a turkey burger from the recipe.**

```
a1
```

```
## # A tibble: 4 x 2
##   type          ‘sum(calories)‘
##   <chr>                   <dbl>
## 1 Meat                     15
## 2 Sauce                    35
## 3 Vegetables                3.5
## 4 Wheat product            25
```

**1b) Write R code to return all recipes containing both bread and tomato products with their weights on these two ingredients.**

```
b1
```

```
## # A tibble: 3 x 2
##   recipe        sum_bread_and_tomato
##   <chr>                        <dbl>
## 1 Beef Burger                     16
## 2 Sandwich                        17
## 3 Turkey Burger                   14
```

**1c) Write R code to return the total calories of the beef and turkey burgers.**

```
c1
```

```
## # A tibble: 2 x 2
##   recipe        total_calories
##   <chr>                  <dbl>
## 1 Beef Burger             1317
## 2 Turkey Burger            777
```

**1d) Write R code to return all vegetable items sold at either 'W-Mart' or 'Food Warehouse' and display which shop offers the lowest price for each item.**

```
d1
```

```
## # A tibble: 5 x 3
##   food_item minprice shop
##   <chr>        <dbl> <chr>
## 1 Cabbage        1.8 Food warehouse
## 2 Onions         1   W-Mart
## 3 Onions         1   Coco Mart
## 4 Spinach        2.5 Food warehouse
## 5 Tomato         3   W-Mart
```

**1e) Write R code to return the recipes with the total calories that contain "Wheat product".**

```
e1
```

```
## # A tibble: 4 x 2
##   recipe                  total_kcal
##   <chr>                        <dbl>
## 1 Beef Burger                   1317
## 2 Sandwich                       562
## 3 Spaghetti and Meatballs       1579
## 4 Turkey Burger                  777
```

## Section 2: Regular Expression Exercises

**Part 1: Find names.txt and write regex patterns to answer following questions.**

**1a) Find all usernames that contain at least one numeric character.**

```
txt <- readLines("./names.txt")
txt
```

```
##  [1] "Abc One23"         "Horrible turtle"   "Messsages"
##  [4] "keep_it_simple"    "hello world!"      "Zoran D Wang"
##  [7] "myUsernameis210"   "abc defg"          "asml"
## [10] "john"              "Edward Lazowska"   "123fionaFog"
## [13] "Red chihuahua5"    "1"                 "CLEAN"
## [16] "+plus+"            "omaha poshy"       "0maha p0shy"
## [19] "OMAHA POSHY"       "Deacon @ Stephens" "Ajay Ann Bryan"
```

```
str_view(txt, pat_1_a)
```

```
##  [1] | Abc One<2><3>
##  [7] | myUsernameis<2><1><0>
## [12] | <1><2><3>fionaFog
## [13] | Red chihuahua<5>
## [14] | <1>
## [18] | <0>maha p<0>shy
```

**1b) Find all usernames that are exactly four characters long and consist only of alphabetic characters.**

```
str_view(txt, pat_1_b)
```

```
##  [9] | <asml>
## [10] | <john>
```

**1c) Find all usernames following the conventional name format, where the given name precedes the family name, with optional additional names in-between. Each name segment is separated by a single white space and should begin with an uppercase letter followed by one or more lowercase letters. Note that intermediate names should also start with an uppercase letter followed by zero or more lowercase letters.**

```
str_view(txt, pat_1_c)
```

```
##  [6] | <Zoran D Wang>
## [11] | <Edward Lazowska>
## [21] | <Ajay Ann Bryan>
```

**Part 2: Find cards.txt and write the regex patterns to identify all the valid card numbers with the correct formatting rules:**

- A Master card number begins with a 5 and it is exactly 16 digits long.
- A Visa card number begins with a 4 and it is between 13 and 16 digits long

```
cards <- readLines("./cards.txt")
cards
```

```
##  [1] "5123456789101112"         "4789 0123 8910 1112"
##  [3] "4444 9321 1230 3"         "5315 4011 1721 51"
##  [5] "4987 9381 2457"           "4891 0870 8908 70987"
##  [7] "5234 4567 8910 1112"      "58907890782309171"
##  [9] "3008 9078 1891 7890"      "5192 9295 91828818"
## [11] "4182 2884 1232 9582 2182"
```

**2a) Write a regex pattern to match valid Master card number and print all the valid numbers, grouped into sets of 4 separated by a single space.**

```
# Valid master card numbers
str_view(cards, pat_2_a)
```

```
##  [1] | <5123456789101112>
##  [7] | <5234 4567 8910 1112>
## [10] | <5192 9295 91828818>
```

**2b) Write a regex pattern to match valid Visa card number and print all the valid numbers, grouped into sets of 4 separated by a single space.**

```
# Valid Visa card numbers
str_view(cards, pat_2_b)
```

```
## [2] | <4789 0123 8910 1112>
## [3] | <4444 9321 1230 3>
```

**Part 3: Find the passwords.txt and write regex patterns to answer the following questions.**

```
passwords <- readLines("./passwords.txt")
```

```
## Warning in readLines("./passwords.txt"): incomplete final line found on
## './passwords.txt'
```

```
passwords
```

```
##  [1] "1234567"        "12345678"       "Strings78"      "apple007"
##  [5] "1brownie"       "asdfjkl"        "?helloWorld"    "90095"
##  [9] "glhf1234"       "H1234!!ap"      "789afk"         "alllowercase"
## [13] "ALLUPPERCASE"   "missingNumbers"
```

**3a) Write a regex pattern to identify the passwords that satisfies the requirements below.**

- Minimum 8 characters
- Must contain at least one letter
- Must contain at least one digit

```
str_view(passwords, pat_3_a)
```

```
##  [3] | <Strings78>
##  [4] | <apple007>
##  [5] | <1brownie>
##  [9] | <glhf1234>
## [10] | <H1234!!ap>
```

**3b) Write a regex pattern to identify the passwords that satisfies the requirements below**

- Minimum 8 characters
- Must contain at least one uppercase character
- Must contain at least one lowercase character
- Must contain at least one digit
- Must contain at least one punctuation character

```
str_view(passwords, pat_3_b)
```

```
## [10] | <H1234!!ap>
```

**Part 4: Please load wordlists.RData to R and write regular expression patterns which will match all of the values in x and none of the values in y.**

**a) Ranges**

```
load("./wordlists.RData")
```

```
wordlists$Ranges
```

```
## $x
##  [1] "abac"   "accede" "adead"  "babe"   "bead"   "bebed"  "bedad"  "bedded"
##  [9] "bedead" "bedeaf" "caba"   "caffa"  "dace"   "dade"   "daff"   "dead"
## [17] "deed"   "deface" "faded"  "faff"   "feed"
##
## $y
##  [1] "beam"   "buoy"   "canjac" "chymia" "corah"  "cupula" "griece" "hafter"
##  [9] "idic"   "lucy"   "martyr" "matron" "messrs" "mucose" "relose" "sonly"
## [17] "tegua"  "threap" "towned" "widish" "yite"
```

```
str_view(wordlists$Ranges$x, pat_4_a)
```

```
##  [1] | <abac>
##  [2] | <accede>
##  [3] | <adead>
##  [4] | <babe>
##  [5] | <bead>
##  [6] | <bebed>
##  [7] | <bedad>
##  [8] | <bedded>
##  [9] | <bedead>
## [10] | <bedeaf>
## [11] | <caba>
## [12] | <caffa>
## [13] | <dace>
## [14] | <dade>
## [15] | <daff>
## [16] | <dead>
## [17] | <deed>
## [18] | <deface>
## [19] | <faded>
## [20] | <faff>
## ... and 1 more
```

```
cat("-------------BREAK---------------\n")
```

```
## -------------BREAK---------------
```

```
str_view(wordlists$Ranges$y, pat_4_a)
```

**b) Backrefs**

```
wordlists$Backrefs
```

```
## $x
##  [1] "allochirally"    "anticovenanting" "barbary"         "calelectrical"
##  [5] "entablement"     "ethanethiol"     "froufrou"        "furfuryl"
##  [9] "galagala"        "heavyheaded"     "linguatuline"    "mathematic"
## [13] "monoammonium"    "perpera"         "photophonic"     "purpuraceous"
## [17] "salpingonasal"   "testes"          "trisectrix"      "undergrounder"
## [21] "untaunted"
##
## $y
##  [1] "anticker"        "corundum"        "crabcatcher"     "damnably"
##  [5] "foxtailed"       "galvanotactic"   "gummage"         "gurniad"
##  [9] "hypergoddess"    "kashga"          "nonimitative"    "parsonage"
## [13] "pouchlike"       "presumptuously"  "pylar"           "rachioparalysis"
## [17] "scherzando"      "swayed"          "unbridledness"   "unupbraidingly"
## [21] "wellside"
```

```
str_view(wordlists$Backrefs$x, pat_4_b)
```

```
##  [1] | <allochirally>
##  [2] | <anticovenanting>
##  [3] | <barbary>
##  [4] | <calelectrical>
##  [5] | <entablement>
##  [6] | <ethanethiol>
##  [7] | <froufrou>
##  [8] | <furfuryl>
##  [9] | <galagala>
## [10] | <heavyheaded>
## [11] | <linguatuline>
## [12] | <mathematic>
## [13] | <monoammonium>
## [14] | <perpera>
## [15] | <photophonic>
## [16] | <purpuraceous>
## [17] | <salpingonasal>
## [18] | <testes>
## [19] | <trisectrix>
## [20] | <undergrounder>
## ... and 1 more
```

```
cat("-------------BREAK---------------\n")
```

```
## -------------BREAK---------------
```

```
str_view(wordlists$Backrefs$y, pat_4_b)
```

### c) Prime

```
wordlists$Prime
```

```
## $x
##  [1] "xx"
##  [2] "xxx"
##  [3] "xxxxx"
##  [4] "xxxxxxx"
##  [5] "xxxxxxxxxxx"
##  [6] "xxxxxxxxxxxxx"
##  [7] "xxxxxxxxxxxxxxxxx"
##  [8] "xxxxxxxxxxxxxxxxxxx"
##  [9] "xxxxxxxxxxxxxxxxxxxxxxx"
## [10] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [11] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [12] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [13] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [14] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [15] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [16] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [17] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [18] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

```
## [19] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
## [20] "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
##
## $y
##  [1] "xxxx"                              "xxxxxx"
##  [3] "xxxxxxxx"                          "xxxxxxxxx"
##  [5] "xxxxxxxxxx"                        "xxxxxxxxxxx"
##  [7] "xxxxxxxxxxxxx"                     "xxxxxxxxxxxxxx"
##  [9] "xxxxxxxxxxxxxxx"                   "xxxxxxxxxxxxxxxx"
## [11] "xxxxxxxxxxxxxxxxx"                 "xxxxxxxxxxxxxxxxxx"
## [13] "xxxxxxxxxxxxxxxxxxx"               "xxxxxxxxxxxxxxxxxxxx"
## [15] "xxxxxxxxxxxxxxxxxxxxx"             "xxxxxxxxxxxxxxxxxxxxxx"
## [17] "xxxxxxxxxxxxxxxxxxxxxxx"           "xxxxxxxxxxxxxxxxxxxxxxxx"
## [19] "xxxxxxxxxxxxxxxxxxxxxxxxx"         "xxxxxxxxxxxxxxxxxxxxxxxxxx"
```

```
str_view(wordlists$Prime$x, pat_4_c)
```

```
##  [1] | <xx>
##  [2] | <xxx>
##  [3] | <xxxxx>
##  [4] | <xxxxxxx>
##  [5] | <xxxxxxxxxx>
##  [6] | <xxxxxxxxxxx>
##  [7] | <xxxxxxxxxxxxx>
##  [8] | <xxxxxxxxxxxxxxx>
##  [9] | <xxxxxxxxxxxxxxxxx>
## [10] | <xxxxxxxxxxxxxxxxxxx>
## [11] | <xxxxxxxxxxxxxxxxxxxxx>
## [12] | <xxxxxxxxxxxxxxxxxxxxxxx>
## [13] | <xxxxxxxxxxxxxxxxxxxxxxxxx>
## [14] | <xxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [15] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [16] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [17] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [18] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [19] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
## [20] | <xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx>
```

```
cat("-------------BREAK---------------\n")
```

```
## -------------BREAK---------------
```

```
str_view(wordlists$Prime$y, pat_4_c)
```